

- 1) What an OS does? $\Rightarrow$ Principles
- 2) How does it perform its actual actions?
 $\Rightarrow$ Latest version of the Linux kernel

Process Scheduler

Schedulers can be of two types $\rightarrow$ 1) Normal
2) Real-Time scheduler

$\Rightarrow$ Ask for some specific

Soft real-time schedulers $\rightarrow$ (Playing video)
Latency guarantees

Hard real-time schedulers $\rightarrow$ (Industries, autonomous cars, etc.)

$\Downarrow$
Linux kernel cannot satisfy these constraints.

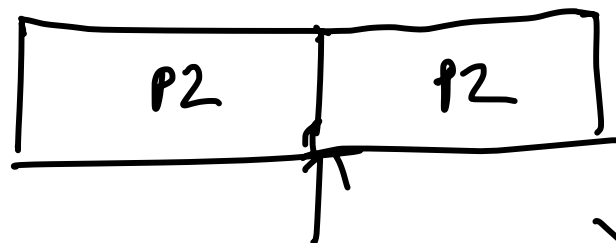
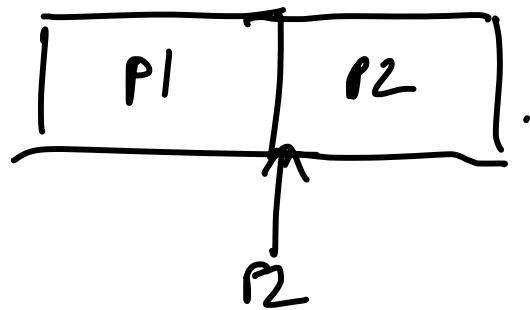
How can the Linux kernel handle both soft real-time constraints and normal jobs together?

- Every process must belong to a scheduling class
- Scheduling class defines the scheduling policy that will be used.
- Each class assumes that processes/threads are preemptive

1) **SCHED_FIFO** → Uses FIFO scheduling
 (Modified version of FIFO) with no timeslice; if higher priority job in this class comes, then this job is moved back to the ready state

priority = 50 (P1)
 = 60 (P2)

[Priority values from 0 to 99; higher values denote stronger priority]



2) **SCHED_RR** (Soft real-time) → Very similar to **SCHED_FIFO**; but with a timeslice of 10ms after which a process is preempted.

3) SCHED_NORMAL (Normal user processes) → Priority values of 0; but "nice" values can be used to change priority.

Nice values → Higher values imply lower priority.
→ -20 to +19
default is 0.

A user can increase the nice value to indicate that the process can be slower than usual.
Only root user can set nice values to negative, not ordinary users.

2.6.23 of the kernel: → Completely Fair Scheduler (CFS)

Provide equal amount of access to the processor to each task

100 ms

50 ms

50 ms

10 ms

...

10 ms

Too many process switches are possible if a fully fair scheduler is used. To avoid it, a floor of 1 ms is kept below which the

Time-slice is not divided.

"Amount of access" $\rightarrow$ Time spent at the processor (stored in a variable called $vruntime$).

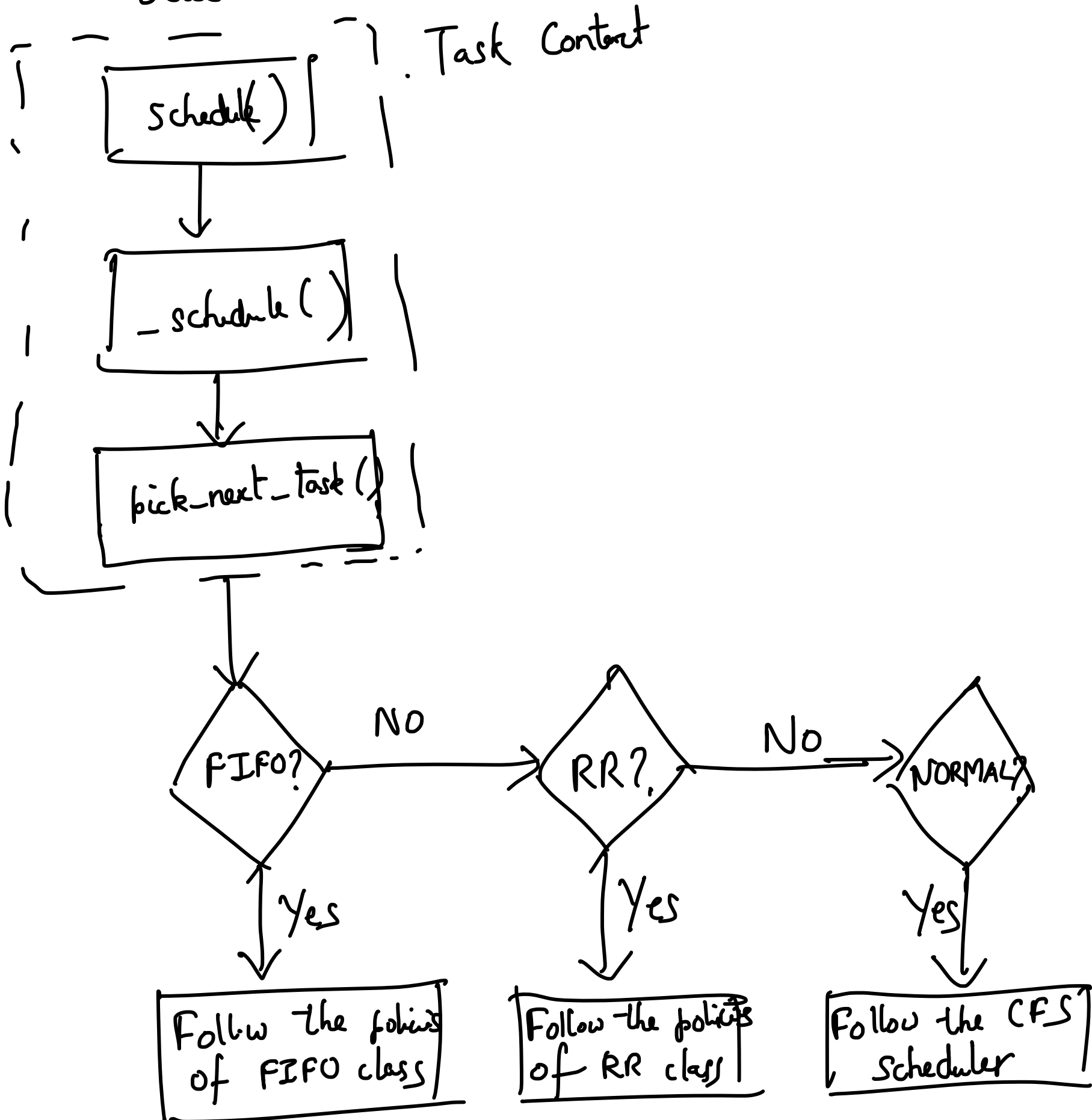
At each point of time, choose the process with the lowest $vruntime$. If kept in a linked list/array, it can take $O(n)$ time. Red-black tree is used to store the $vruntimes$, which reduces the time complexity to $O(\log n)$.

There is a table of weights based on the nice values, where the higher nice values have higher weights

$$\left[\begin{array}{l} vruntime += \text{actual-processor-time} \\ \quad * \text{weight}[\text{nice-value}]; \end{array} \right]$$

System call called "nice" which can be used to change the nice value of a process.
How exactly processes are scheduled across classes?

There is a priority among classes. If a process with higher "class priority" is present, then that process will always be selected.



FIFO and RR classes have their own linked list; their processes are not kept in the red-black tree.

CFS eliminates the need to separately classify processes based on interactive or non-interactive.

What can be done if there are multiple cores?

→ Single "runqueue" approach → Any processor that becomes free accesses the data structure and removes a task. Concurrent access can only happen via locking mechanism. If there are many processors, then multiple of them may have to wait for the lock to be released, which is wastage of processors time.

→ Individual "runqueue" approach → Each processor has its own set of processors; and the tasks are distributed among the ~~processors~~ processors. An idle processor would check the runqueues of other processors, and if found

non-empty, it will take some of the tasks of those processors.

If you want a process to stick to a single processor (i.e. disable work stealing to take away this process), then set the processor affinity of a processor.

Processor affinity of a process = processor #.
There is a specific system call to set it.

Number of data structures present to schedule tasks = NO of scheduling classes $\times$ Number of processors